

Sprint: Expected Result Excellence

Branch: `expected-result-excellence` • **Requirement under test:** Add Employee (only) • **Discipline:** one defect, fixed and proven before/after — no new engines, prompts, or architecture.

Mandate (verbatim):

Fix `buildExpected()` so that it generates business-observable assertion lists rather than generic success statements. Use only information already available from the planner, requirement, and application profile. Do not introduce new engines, prompts, or architecture. Regenerate the Add Employee suite and prove that every Expected Result is something a Senior QA would accept without rewriting.

Bar for done: a QA Lead approves every Expected Result **without editing it**.

Phase 1 — Audit: what the Expected Results looked like

I regenerated the full 39-scenario Add Employee suite and read every Expected Result. They collapsed onto **six generic strings**. The Expected Result was a placeholder, not an assertion — it did not say **what changed, what stayed unchanged, what became visible/persistent, or which business rule was enforced**.

Scenario	Old Expected Result
<code>crud-pos-create</code> (Create with valid data)	"The action succeeds and the user reaches the expected next state, with confirmation shown."
<code>crud-neg-duplicate</code> (Duplicate employee ID)	"The action is rejected, a clear, specific error message is shown, and no state change or navigation occurs."
<code>crud-neg-direct-endpoint-authz</code> (Authorization)	(identical to duplicate) "The action is rejected, a clear, specific error message is shown, and no state change or navigation occurs."
<code>crud-neg-injection-xss</code> (XSS payload)	(identical to duplicate) "The action is rejected, a clear, specific error message is shown, and no state change or navigation occurs."
<code>field-first-name-max-accepted</code> (boundary accepted)	"Values at and within the limit are accepted; values beyond the limit are rejected with a clear boundary/validation message."
<code>field-first-name-over-max</code> (boundary rejected)	(identical to accepted) "Values at and within the limit are accepted; values beyond the limit are rejected with a clear boundary/validation message."

The two worst symptoms:

1. **Opposite scenarios shared one string.** The boundary-accepted case and the boundary-rejected case had **byte-for-byte identical** Expected Results — the reader cannot tell what the test is supposed to prove.
2. **Every negative was the same sentence.** Duplicate, authorization, and injection — three completely different business rules — all read "The action is rejected...". No QA would sign off on any of them without rewriting.

Phase 2 — Root cause

File: `src/engines/scenario-builder.ts` • **Function:** `buildExpected()`

`buildExpected()` switched **only on** `coverageType` and returned **one hard-coded sentence per type**. Because 39 scenarios map onto ~6 coverage types, they collapsed onto ~6 strings. The function never looked at what the scenario actually was (create vs duplicate vs authz vs injection), nor at the real entity/fields it operated on — even though the planner scenario, the requirement, and the application profile already carried all of that.

So the information to write a proper assertion list was **already in hand** — `buildExpected()` simply threw it away.

Phase 3 — The change (data-shape only, no new architecture)

One function was reshaped. It now derives a **list of concrete, black-box assertions** from data that already exists:

- **The requirement** → the business entity. `deriveEntity("Add Employee")` → **"Employee"** (strips the CRUD verb/article; falls back to "record" when nothing is derivable).
- **The application profile** → the destination and the fields. `deriveListName()` → **"Employees list"** (from the profile's list page); `enteredFieldsPhrase()` → **"First Name, Last Name and Employee ID"**; `identifierFieldLabel()` → **"Employee ID"**.
- **The planner scenario** → the intent. The existing `coverageType` plus the scenario's own object-ive/title are used to sub-classify negatives (duplicate vs validation vs authorization vs injection) and boundaries (accept vs reject) — reading only fields the planner already emits.

Each branch returns a **checklist** via a single `finalize()` helper. The Expected Result is now a first-class list:

- `expected.assertions: string[]` — the canonical, machine-readable assertion list (new field).
- `expected.observable: string` — the same list rendered as a ✓-prefixed checklist (this is what every renderer and the CSV/XLSX export already read, so **no renderer or exporter changed**).

Guardrails honoured: no new engine, prompt, planner field, or file. `buildExpected()` consumes only the planner output, the requirement, and the profile. Polarity is deliberately clean (positive lists read as success, negative lists carry a rejection/denial token) so the existing `expected_result_consistency` integrity gate never downgrades a case.

Phase 4 — Before / After (regenerated Add Employee suite)

Positive — Create with valid data (`crud-pos-create`)

Before: "The action succeeds and the user reaches the expected next state, with confirmation shown."

After:

1. The Employee record is created successfully.
2. A success confirmation message is displayed.
3. The entered First Name, Last Name and Employee ID values are saved exactly as entered.
4. The new Employee appears in the Employees list.
5. After saving, the user is taken to the Employee details or the updated Employees list.
6. The Employee record persists after a page refresh (it is durably saved).

Negative — Duplicate employee ID (`crud-neg-duplicate`)

Before: "The action is rejected, a clear, specific error message is shown, and no state change or navigation occurs."

After:

1. The duplicate is rejected — a second Employee record is NOT created.
2. A clear "already exists" uniqueness error is shown, identifying the conflicting Employee ID.
3. The original existing Employee is left unchanged.
4. The total Employee count does not increase.

Negative — Authorization (crud-neg-direct-endpoint-authz)

Before: (same generic string as duplicate)

After:

1. Access is denied — the Employee operation is not permitted for this user.
2. No Employee record is created or modified.
3. The user sees an access-denied / not-authorised message (or is redirected to login).
4. The block is enforced server-side, not merely hidden in the UI.

Negative — XSS payload (crud-neg-injection-xss)

Before: (same generic string as duplicate)

After:

1. The malicious input is rejected or safely neutralised — stored/escaped as literal text, never executed or interpreted.
2. No script runs and no injected markup is reflected back in the UI or any Employee view.
3. The Employee is either not created, or created with the payload safely escaped — no data corruption.
4. A safe, generic error is shown — no stack trace, SQL, or sensitive detail leaks.

Boundary — First Name at max length, accepted (field-first-name-max-accepted)

Before: “Values at and within the limit are accepted; values beyond the limit are rejected with a clear boundary/validation message.”

After:

1. The boundary First Name value is accepted.
2. The Employee is created successfully with the boundary value.
3. The value is stored exactly as entered — no truncation, trimming, or modification.
4. The saved Employee displays and is retrievable with the exact value intact.

Boundary — First Name over max length, rejected (field-first-name-over-max)

Before: (byte-for-byte identical to the accepted case above)

After:

1. The over-limit First Name value is rejected.
2. A clear length/validation message is shown, stating the allowed maximum.
3. No Employee record is created.
4. The entered data is retained so the user can correct it.

The two boundary cases that used to be **identical** are now unmistakably opposite — one asserts accepted + stored intact, the other rejected + nothing created.

Field-scoped negative — First Name whitespace-only (field-first-name-whitespace)

Before: “The action is rejected, a clear, specific error message is shown...”

After:

1. The Employee is NOT created — no record is saved.
 2. A clear, specific validation error is shown **for the First Name field**.
 3. The form stays on screen with the entered values retained for correction.
 4. No partial or malformed Employee record is persisted.
-

Verification

- `npx tsc --noEmit` — clean.
- **Regression suites** (`scenario-builder` , `scenario-correctness` , `scenario-integrity` , `qa-knowledge-stepflow` , `scenario-planner` , `scenario-planner-field-aware` , `scenario-planner-standard-coverage`) — **154 tests green**, no polarity-gate downgrades.
- **New net** — `tests/unit/expected-result-excellence.test.ts` , **19 tests**, asserts mechanically that: every Expected Result is a ≥ 2 -item assertion list; the `observable` is a ✓ checklist mirroring it; positive-create names the entity + all three fields + the list + persistence; duplicate/validation/authorization/injection/boundary-accept/boundary-reject each produce a different, intent-matching list; field-scoped negatives name the specific field; and polarity is clean.
- **Full suite audit** — all 39 Add Employee Expected Results were re-read; every one is now a business-observable checklist. No scenario falls back to a generic one-liner.

Scope honoured / not touched

- No new engine, prompt, planner field, renderer, or exporter. One function reshaped; one optional `assertions` field added to the expected object.
- Add Employee only. Checkout / Banking / CRM / HRMS untouched.

Next candidate defect (not this sprint)

Test **Data** is still generic (“Use data appropriate to the scenario...”). That is the highest-value remaining gap — scenario-specific data derived from the same planner/profile data, one defect at a time.